✅ **Here's everything you asked for** — clean, ready-to-use implementations.

I'll deliver:

1. **Full updated files** (`displayColors.ts + DisplayLine` type + example page renderer)
2. **Canvas-based high-fidelity Display component** (recommended for best visuals)
3. **Color Mapping Guide**

---

1. Updated Files

A. `shared/src/fmc/displayColors.ts` **(Refined)**

```
TypeScript

// shared/src/fmc/displayColors.ts
export type BoeingColor =
  | 'cyan'
  | 'green'
  | 'magenta'
  | 'white'
  | 'shaded'
  | 'amber'
  | 'red';

export type AirbusColor =
  | 'white'
  | 'blue'
  | 'green'
  | 'amber'
  | 'magenta'
  | 'cyan';

export type DisplayColor = BoeingColor | AirbusColor;

export const COLOR_CLASSES: Record<DisplayColor, string> = {
  cyan: 'text-cdu-cyan',
  green: 'text-cdu-green',
  magenta: 'text-cdu-magenta',
  white: 'text-cdu-white',
  shaded: 'text-cdu-shaded',
  amber: 'text-cdu-amber',
```

```
  red: 'text-cdu-red',
  blue: 'text-cdu-blue',
};

export const COLOR_HEX: Record<DisplayColor, string> = {
  cyan: '#00d0ff',
  green: '#39ff14',
  magenta: '#e879f9',
  white: '#ffffff',
  shaded: '#e5e5e5',
  amber: '#ffb000',
  red: '#ff3333',
  blue: '#60a5fa',
};

export const BOEING_DEFAULT_COLOR: DisplayColor = 'green';
export const AIRBUS_DEFAULT_COLOR: DisplayColor = 'amber';

export function getColorClass(color?: DisplayColor): string {
  if (!color) return COLOR_CLASSES.white;
  return COLOR_CLASSES[color] ?? COLOR_CLASSES.white;
}

export function getColorHex(color?: DisplayColor): string {
  if (!color) return COLOR_HEX.white;
  return COLOR_HEX[color] ?? COLOR_HEX.white;
}

export function isValidColor(color: string): color is DisplayColor {
  return color in COLOR_CLASSES;
}
```

B. Updated `DisplayLine` Type

Add this to `shared/src/types/fmc.ts`:

```
TypeScript
```

```
export interface DisplayLine {
  text: string;
  leftLabel?: string;
  rightLabel?: string;

  color?: DisplayColor;
  inverse?: boolean;
  small?: boolean;
  blinking?: boolean;
  modified?: boolean;           // ← New: easier handling of shaded text
  semantic?: DisplaySemantic;
}
```

C. Example Page Renderer (Updated)

Here's how to use the new system in a page (example: `shared/src/fmc/pages/ident.ts`):

```
TypeScript

import type { DisplayLine } from '../../types/fmc';

export function renderIdentPage(state: any): DisplayLine[] {
  return [
    { text: 'IDENT', color: 'cyan' },
    { text: 'MODEL', color: 'white', leftLabel: '1/2' },
    { text: 'B737-800', color: 'green' },
    { text: 'ENG RATING', color: 'white' },
    { text: 'CFM56-7B', color: 'green' },
    { text: 'NAV DATA', color: 'white' },
    { text: 'JUL23', color: 'green' },
    { text: 'OP PROGRAM', color: 'white' },
    { text: '2247662-03', color: 'green' },
    { text: '', color: 'white' },
    { text: 'FMC21A1', color: 'shaded', modified: true },
  ];
}
```

---

2. Canvas-Based High-Fidelity Display Component (Recommended)

For the best visual quality (closest to real hardware), I recommend using a `<canvas>` for the display.

src/components/CDU/CduDisplayCanvas.tsx

```tsx
import React, { useRef, useEffect } from 'react';
import type { DisplayLine } from '@virtual-cdu/shared';
import { getColorHex } from '@virtual-cdu/shared/src/fmc/displayColors';

interface CduDisplayCanvasProps {
  lines: DisplayLine[];
  width?: number;
  height?: number;
}

export const CduDisplayCanvas: React.FC<CduDisplayCanvasProps> = ({
  lines,
  width = 480,
  height = 320,
}) => {
  const canvasRef = useRef<HTMLCanvasElement>(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;

    const ctx = canvas.getContext('2d', { alpha: true });
    if (!ctx) return;

    // Clear
    ctx.fillStyle = '#0a0a0a';
    ctx.fillRect(0, 0, width, height);

    // Font settings (tuned for realism)
    ctx.font = '500 15px "Courier New", monospace';
    ctx.textBaseline = 'top';

    const lineHeight = 18;
    const startY = 8;

    lines.forEach((line, index) => {
      const y = startY + index * lineHeight;
      const color = getColorHex(line.color);
```

```
      ctx.fillStyle = color;
      ctx.fillText(line.text.toUpperCase(), 12, y);

      // Optional: Draw left/right labels
      if (line.leftLabel) {
        ctx.fillText(line.leftLabel, 12, y);
      }
      if (line.rightLabel) {
        ctx.fillText(line.rightLabel, width - 80, y);
      }
    });

    // Optional: Light scanline effect
    ctx.strokeStyle = 'rgba(255,255,255,0.04)';
    ctx.lineWidth = 1;
    for (let y = 0; y < height; y += 4) {
      ctx.beginPath();
      ctx.moveTo(0, y);
      ctx.lineTo(width, y);
      ctx.stroke();
    }
  }, [lines, width, height]);

  return (
    <canvas
      ref={canvasRef}
      width={width}
      height={height}
      className="rounded border border-gray-700 bg-[#0a0a0a]"
    />
  );
};
```

Advantages of Canvas version:

- Pixel-perfect control
- Much easier to add scanlines, slight glow, or custom character spacing
- Better performance for complex effects

---

3. Color Mapping Guide

Here's a practical guide for which color to use:

Data Type	Recommended Color	Example Use Case	Notes
Page Titles	`cyan`	`IDENT` , `LEGS` , `RTE`	Most page headers
Active Guidance / Target	`magenta`	Active waypoint, target altitude	Most important data
VOR / Navaid Data	`green`	VOR frequency, radial	Navigation aids
Normal / Entered Data	`white`	Airport codes, speeds, altitudes	Default for most data
Modified Data (EXEC lit)	`shaded`	Data that will be executed	Use `modified: true` too
Inactive / Placeholder	`shaded` or `white`	`----` , `....`	Lower emphasis
Warnings / Errors	`red`	Error messages	Use sparingly
Airbus specific	`amber` / `blue`	Airbus pages	Use Airbus colors on Airbus mode

Quick Rules of Thumb:

- **Cyan** → Titles
- **Magenta** → Active / flying towards
- **Green** → VORs and some active values
- **White** → Everything else
- **Shaded** → Data waiting to be executed

Next Steps Recommendation

1. Replace `displayColors.ts` with the updated version above.
2. Update the `DisplayLine` interface.

3. Start by updating **3–4 key pages** (IDENT, LEGS, PERF INIT, RTE) using the color guide.
4. Replace your current Display component with the **Canvas version** (biggest visual upgrade).
5. Add the Tailwind colors I showed earlier.

Would you like me to also provide:

- A full example of updating one complete page (e.g. `LEGS`) with colors?
- The Tailwind config changes + global CSS for the canvas display?
- A version that supports both **DOM** and **Canvas** rendering (with a toggle)?

Just say the word and I'll deliver the next piece.